

Skripta: IoT a vstavané systémy

Kolektív

4. júna 2026

Obsah

1	Úvod	3
2	Úvod	4
3	Vstavané systémy — podrobný prehľad	6
3.1	Architektúra systému	6
3.2	Architektonický návrh a rozdelenie zodpovedností	6
3.3	Výber procesorovej platformy	6
3.4	Časovanie, plánovanie a deterministické správanie	6
3.5	Verifikácia a testovanie	7
3.6	Regulačné a normatívne aspekty	7
3.7	EMC/EMI a fyzické návrhy	7
3.8	Napájanie a energetická efektívnosť	7
3.9	Bezpečnostné vzory	7
3.10	Nasadenie a životný cyklus produktu	7
3.11	Zhrnutie	7
4	IoT, siete a topológie	8
4.1	IoT architektúra — vrstvy a komponenty	8
4.2	Sieťové topológie	8
4.3	Praktické scenáre a odôvodnenie	9
4.4	Komunikačné protokoly	10
4.5	Edge vs. cloud processing	10
4.6	Brána (gateway) — funkcie	10
4.7	Škálovanie a správa	10
4.8	Bezpečnostné požiadavky	10
4.9	Príklady nasadení	10
4.10	Zhrnutie	11
5	Komunikačné protokoly	12
5.1	MQTT	12
5.2	CoAP	12
5.3	HTTP/REST a WebSocket	12
5.4	Narrowband a LPWAN technológie: LoRaWAN, NB-IoT, Sigfox	13
5.5	Bluetooth Low Energy (BLE) a Zigbee	13
5.6	6LoWPAN, Thread a IPv6	13
5.7	Bezpečnostné aspekty protokolov	13
5.8	Kritériá výberu protokolu	13
6	ESP32 – prehľad platformy	14

7	MicroPython pre vstavané systémy	16
8	Príklady a experimentálne cvičenia	18
9	Servisný test dosky	18
10	Metodika experimentov a hodnotenie	21
11	Interakcia s tlačidlami: hra hádania	21
12	Digitálne riadenie RGB LED	23
13	PWM riadenie jasů RGB LED	24
14	Prerušenia a meranie dĺžky stlačenia	25
15	Sériové LED WS2812B	26
16	PWM bzučiak a krátka melódia	27
17	Analógové meranie svetla	28
18	Senzor DHT: teplota a vlhkosť	29
19	Wi-Fi prístupový bod a webové ovládanie	30
20	Odosielanie dát do ThingSpeak	32
21	MQTT publish/subscribe komunikácia	33

1 Úvod

Tento text predstavuje systematický a didakticky usporiadaný prehľad problematiky Internetu vecí (Internet of Things, IoT) a vstavaných (embedded) systémov so zameraním na návrh, implementáciu a praktické overenie riešení. Skripta kombinuje akademicky podložené teoretické východiská s konkrétnymi praktickými príkladmi a experimentálnymi postupmi, ktoré sú vhodné pre použitie vo výučbe aj pri samostatnom štúdiu.

Cieľom je poskytnúť čitateľovi:

- jasné pochopenie základných konceptov a architektúr vstavaných systémov;
- detailný prehľad komunikačných mechanizmov a protokolov bežne využívaných v IoT (vrátane ich vlastností, obmedzení a návrhových kompromisov);
- praktické návody pre implementáciu a ladenie riešení na reálnych platformách (s dôrazom na platformu ESP32 a prostredie MicroPython);
- súbor príkladov a cvičení umožňujúcich samostatné overenie a rozšírenie poznatkov.

Skriptum je koncipované pre študentov technických a prírodovedných programov, ako aj pre vývojárov prototypov a inžinierov, ktorí chcú získať systematický prehľad o metódach návrhu a praktických aspektoch nasadzovania IoT riešení. Predpokladá sa základná znalosť programovania (najmä jazykov Python alebo C) a elementárne znalosti elektroniky; tam, kde sú potrebné, sú základné pojmy kratko rekapitulované.

Metodologicky sa skripta opiera o kombináciu nasledovných prístupov: formálne definície a kategorizácie, komparatívna analýza alternatívnych technológií, návrhové vzory pri implementácii a praktické príklady s overiteľnými výsledkami. Autori kladú dôraz na to, aby boli vysvetlené dôvody návrhových rozhodnutí, nie iba opis postupov.

Organizácia textu: po úvodnom prehľade nasledujú kapitoly rozvíjajúce problematiku postupne do väčšej hĺbky. Najprv sú predstavené architektúry vstavaných systémov a základné hardvérové komponenty, potom sú rozpracované komunikačné topológie a protokoly, na ktoré nadväzujú praktické kapitoly venované konkrétnej platforme ESP32 a programovaniu v MicroPython. Záver tvorí súbor príkladov a cvičení s komentovanými riešeniami.

V rámci textu sú explicitne uvedené predpoklady a rozsah jednotlivých kapitol. Niektoré témy (napr. hlboká kybernetická bezpečnosť IoT, pokročilé cloudové integrácie alebo optimalizácia na úrovni architektúr SoC) sú zmienené ako odporučená literatúra a predmet ďalšieho štúdia, avšak nie sú v tomto skripte rozpracované do úplnej hĺbky.

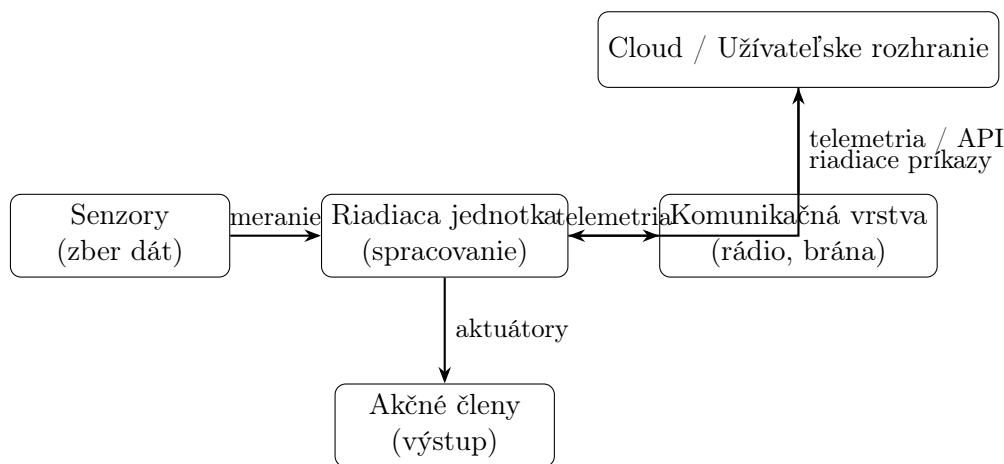
Použitá terminológia a konvencie: tam, kde je to vhodné, používame formálne definície a jednotné označenie veličín. V kódoch a príkladoch sú uvedené jasné komentáre a referenciačné poznámky. Všetky zdroje a literatúra sú uvedené v bibliografii.

Záverom autori dúfajú, že táto skripta poslúži ako robustný východiskový materiál pre ďalšie akademické i praktické aktivity v oblasti IoT a vstavaných systémov.

2 Úvod

Táto kapitola poskytuje systematický prehľad základných konceptov vstavaných systémov s dôrazom na ich aplikácie v kontexte Internetu vecí. Definujeme vstavaný systém ako samostatnú výpočtovú jednotku, ktorá je špeciálne navrhnutá na vykonávanie úloh s obmedzeniami v oblasti pamäte, výpočtového výkonu, energetickej spotreby a fyzických rozmerov.

Typická architektúra IoT zariadenia sa skladá z niekoľkých logických vrstiev: (1) zber dát (senzory), (2) lokálne spracovanie a riadenie (mikrokontrolér alebo systém na čipe), (3) komunikačná vrstva (radiové rozhranie a protokoly) a (4) aplikačná/užívateľská vrstva (cloud, vizualizácia, API). Takéto vrstvenie uľahčuje modulárny návrh, testovateľnosť a možnosť opätovného použitia komponentov.



Obrázok 1: Bloková schéma typického IoT zariadenia.

Kľúčové aspekty návrhu možno rozdeliť do dvoch zorných uhlov: hardvérového a softvérového. V hardvéri ide najmä o výber mikrokontroléra, periférií, napájacieho reťazca a komunikačných rozhraní. V softvéri ide o architektúru firmvéru, plánovanie úloh, spracovanie prerušení, manažment pamäte a zabezpečenie.

Hardvérové prvky

Mikrokontrolér (MCU) je centrálny komponent, no kompletný systém zahŕňa senzory, aktuátory, pasívne a aktívne ochranné obvody, kondenzátory pre vyhladzovanie napätia a mechanické prvky. Dôležitými perifériami sú ADC (analogovo-digitálne prevodníky), DAC (digitálno-analogové prevodníky), PWM (generovanie pulsačnej modulácie), časovače, sériové rozhrania (I2C, SPI, UART) a špecializované moduly ako CAN, USB alebo RMT.

Pri výbere sú dôležité metriky: výkon na spotrebované jadro (energy per operation), dostupná RAM a flash, počet a typ periférií, podpora nízkoenergetických režimov a podpora požadovaných rádiových rozhraní (Wi-Fi, BLE, LoRa atď.).

Softvérové modely

Firmvér môže byť implementovaný buď v forme jednoduchej hlavnej slučky (superloop), alebo nad RTOS (Real-Time Operating System). Superloop je jednoduchý a predvídateľný, vhodný pre malé projekty; RTOS prináša plánovanie priorít, synchronizáciu úloh a deterministické správanie potrebné pri hard-real-time požiadavkách.

Pri návrhu softvéru je potrebné venovať osobitnú pozornosť správe prerušení (ISR by mali byť čo najkratšie), ochrane kritických sekcií (mutexy, semaforey), alokácii pamäte (minimalizovať dy-

namickú alokáciu v kritickom kóde) a mechanizmom obnovy po chybách (watchdog, persistencia stavu).

Výber rozhraní a periférií

Voľba medzi I2C, SPI, UART, I2S a ďalšími rozhraniami závisí od rýchlostných požiadaviek, vzdialenosti medzi komponentmi, počtu zariadení na zbernici a citlivosti na rušenie. Výber tiež ovplyvňuje softvérová komplexita: napríklad SPI môže vyžadovať vyšší počet vodičov a logiku pre výber čipu, avšak poskytuje vyššiu priepustnosť.

V priemyselných aplikáciách sa často využíva CAN/TWAI pre jeho robustnosť a deterministické vlastnosti; v audio aplikáciách je preferované I2S pre synchronizáciu vzoriek.

Energetické aspekty a správa napájania

Energetická optimalizácia zahŕňa viacero úrovní: výber efektívnych komponentov, minimalizáciu aktivít CPU, adaptívne správanie rádií (batching, dlhšie intervaly zasielania), a využívanie hlbokých spánkových režimov. Návrh napájacieho reťazca musí riešiť transientné prúdy (napr. pri zapnutí Wi-Fi), filtráciu a ochranu.

Bezpečnosť a spoľahlivosť

Bezpečnostné mechanizmy by mali zahŕňať autentifikáciu zariadení, šifrovanie komunikácie (TLS/DTLS), bezpečné ukladanie kľúčov (napr. v dedikovanom bezpečnostnom module) a zabezpečený mechanizmus aktualizácií firmvéru (signed OTA).

Spoľahlivosť zahŕňa návrh na zvládanie chýb: watchdogy, redundanciu kritických prvkov, logovanie chýb a telemetriu pre diagnostiku. Súčasťou návrhu môže byť aj plán údržby a mechanizmus rollbacku pri neúspešnej aktualizácii.

Záver

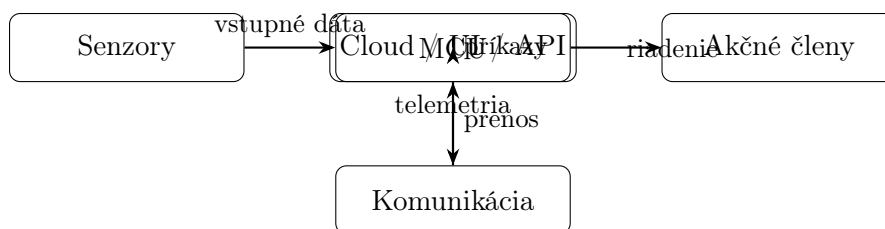
Táto kapitola poskytla koncepčný rámec pre ďalšie, detailnejšie kapitoly, ktoré rozpracujú konkrétne architektonické rozhodnutia, návrh napájania, časovanie a praktickú implementáciu na platforme ESP32.

3 Vstavané systémy — podrobný prehľad

Táto kapitola rozpracováva kľúčové aspekty návrhu vstavaných systémov. Je určená pre inžinierov a študentov, ktorí potrebujú praktické rady pri výbere komponentov, návrhu napájania, riadení časovania a správe softvéru.

3.1 Architektúra systému

Základom návrhu je pochopenie funkčných blokov: senzory, spracovanie, pamäť, komunikácia a akčné členy. Pri menších systémoch postačuje jednoduchá jednovrstvová architektúra, pri zložitejších sa uplatní viacvrstvový prístup s jasným oddelením zberu dát, lokálneho spracovania a odosielania do cloudu.



Obrázok 2: Všeobecná bloková schéma vstavaného IoT zariadenia.

Pri návrhu treba zvážiť:

- požadovaný výpočtový výkon a spotrebu,
- dostupnú pamäť (RAM a flash),

3.2 Architektonický návrh a rozdelenie zodpovedností

Dobrá architektúra oddelí zber dát, lokálne spracovanie, komunikačnú vrstvu a aplikačnú logiku. Pre návrh je užitočné formalizovať požiadavky: latenciu, spoľahlivosť (SLA), energetické limity, bezpečnostné ciele a podmienky prevádzky. Na základe týchto požiadaviek sa volia komponenty a stratégiu rozdelenia výpočtovej záťaže medzi edge a cloud.

3.3 Výber procesorovej platformy

Okrem základných kritérií (CPU, pamäť, periférie) by mal návrh zohľadniť:

- Ekosystémový support (toolchain, debug nástroje, knižnice),
- Dostupnosť bezpečnostných modulov (TPM, secure element),
- Podporu energeticky efektívnych režimov (deep sleep, power domains),
- Certifikačné požiadavky a prevádzkové prostredie (teplota, vlhkosť).

3.4 Časovanie, plánovanie a deterministické správanie

Pri systémoch s real-time požiadavkami je potrebné definovať časové záväzky a garantované latencie. Plánovanie úloh môže byť preemptívne (RTOS) alebo kooperatívne; výber algoritmu plánovania (fixed priority, EDF) závisí od požiadaviek na deterministickosť a jednoduchosti analýzy.

3.5 Verifikácia a testovanie

Okrem tradičných testovacích postupov (jednotkové testy, integračné testy, systémové testy) je dôležité zaviesť:

- návrhové overenie (model checking, statická analýza),
- testovanie za hraníc prevádzkových podmienok (stres, teplota),
- overenie elektromagnetickej kompatibility (EMC/EMI),
- bezpečnostné testy (penetration testing, fuzzing komunikačných rozhraní).

3.6 Regulačné a normatívne aspekty

Pri návrhu produktov pre masové nasadenie treba zohľadniť platné normy (CE, FCC, RED) a priemyselné štandardy. Dôležité sú tiež regulácie týkajúce sa rádiového spektra a environmentálne požiadavky (RoHS).

3.7 EMC/EMI a fyzické návrhy

Dobre navrhnutá doska plošných spojov (PSP) a napájací reťazec minimalizujú emisné a imunitné problémy. Odporúčania zahŕňajú: oddelenie analógových a digitálnych pôd, vhodné rozmiestnenie kondenzátorov, stínovanie a sledovanie impedancií pri vysokofrekvenčných linkách.

3.8 Napájanie a energetická efektívnosť

Okrem komponentov napájacieho reťazca treba navrhnuť stratégiu správy energie: profilovanie spotreby, použitie adaptívnych intervalov zasielania, hibernáciu periférií a optimalizáciu algoritmov z hľadiska energie. Pre batériové systémy je kľúčové navrhnuť meranie stavu batérie a spoľahlivé mechanizmy obnovenia po vybití.

3.9 Bezpečnostné vzory

Odporúčané postupy zahŕňajú: bezpečné zavádzanie kľúčov pri výrobných procesoch, hardvérové zabezpečenie kľúčov (secure element), používanie overených kryptografických knižníc, zabezpečený proces OTA aktualizácií a pravidelnú bezpečnostnú údržbu.

3.10 Nasadenie a životný cyklus produktu

Pri plánovaní nasadenia treba definovať stratégiu aktualizácií, telemetriu pre prevádzku, monitorovanie chýb a plán údržby. Pre rozsiahle nasadenia je dôležité mať mechanizmy pre hromadný provisioning zariadení a bezpečný lifecycle management.

3.11 Zhrnutie

Táto kapitola poskytuje podrobný technologický a procesný rámec pre návrh, verifikáciu a prevádzku vstavaných IoT systémov. Nasledujúce kapitoly aplikujú tieto princípy v konkrétnych implementáciách na platforme ESP32 a v prostredí MicroPython.

4 IoT, siete a topológie

Táto kapitola sa venuje štruktúre IoT systémov, bežným sieťovým topológiám a spôsobom integrácie lokálnych zariadení s bránami (gateway) a cloudovými službami.

4.1 IoT architektúra — vrstvy a komponenty

Typická IoT architektúra zahŕňa nasledujúce vrstvy:

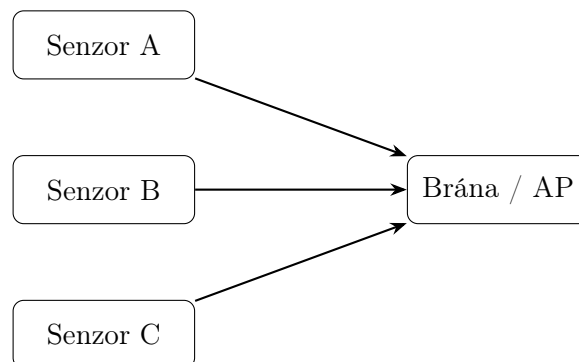
- Periférna vrstva: senzory a akčné členy pripojené k mikrokontrolérom,
- Edge vrstva: lokálne spracovanie, agregácia dát, krátkodobé ukladanie a predspracovanie (filtering, kompresia),
- Brána (gateway): protokolová prekladacia vrstva, zabezpečená konektivita do internetu, lokálna politika a prípadne správa zariadení,
- Cloud/service vrstva: dlhodobé úložisko, analýza dát, UI a integračné API.

4.2 Sieťové topológie

Výber topológie závisí od scenára použitia, fyzických obmedzení a požiadaviek na spoľahlivosť.

Hviezda (star)

Najjednoduchšia topológia, kde všetky zariadenia komunikujú priamo s centrálnou bránou. Výhody: jednoduchá správa, nízka latencia pre priame príkazy. Nevýhoda: single point of failure v prípade zlyhania brány.



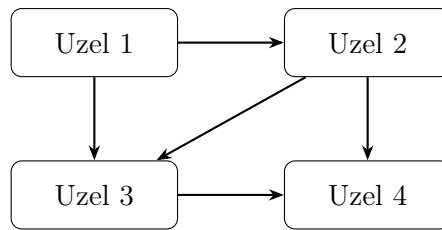
Obrázok 3: Hviezdicová topológia s centrálnou bránou.

Mesh

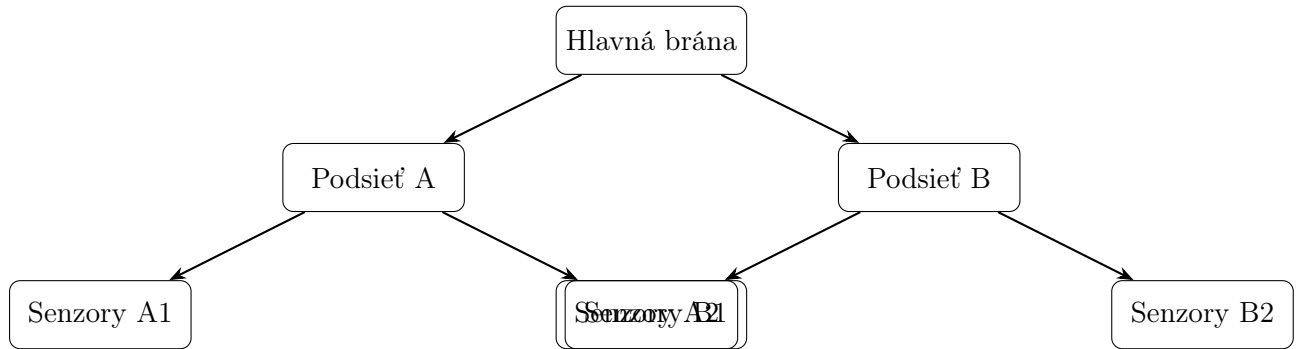
V mesh sieti zariadenia môžu smerovať správy cez viacero uzlov. Výhody: rozšíriteľnosť, odolnosť voči chybám (self-healing). Nevýhody: zložitejšie smerovanie, väčšia spotreba uzlov, komplikovanejšia správa synchronizácie.

Tree / hierarchická

Používa sa pri väčších nasadeniach, kde sú lokalizované podsiete pre efektívne smerovanie a segmentáciu. Umožňuje hierarchickú správu a zníženie broadcastov.



Obrázok 4: Ukážka mesh topológie s viacerými cestami medzi uzlami.



Obrázok 5: Hierarchická topológia s podskupinami zariadení.

Hybridné topológie

V praxi sa často používajú kombinácie, napríklad mesh v lokálnej sieti a hviezda smerom k bráne.

4.3 Praktické scenáre a odôvodnenie

Smart home

V inteligentnej domácnosti je praktická hviezda s centrálnym Wi-Fi AP alebo domácou bránou, pretože zariadenia bývajú v dosahu jednej miestnosti alebo bytu a potrebujú jednoduchú správu. Ak sú senzory na batériu a v dome je viac prekážok, často sa vyplatí Zigbee/Thread mesh, lebo umožní lepšie pokrytie a nižšiu spotrebu než Wi-Fi.

Priemyselná hala

V priemyselnej hale sa oplatí hierarchická alebo hybridná topológia, pretože zariadenia sú rozdelené do zón a medzi zónami bývajú rušenie, dlhšie trasy a rozdielne požiadavky. Lokálne podsiete môžu spracovať dáta v edge uzle a do nadradenej vrstvy posilať iba agregované výsledky, čo znižuje záťaž aj latenciu.

Rozľahlé monitorovanie

Pre poľnohospodárske parcely, environmentálne senzory alebo rozmiestnené meracie body je vhodný LoRaWAN alebo podobná nízkoenergetická hviezda s gateway, pretože zariadenia musia fungovať mesiace až roky na batériu. Tu je dôležitejší dosah a výdrž než vysoká priepustnosť, preto sa dáta posilajú len v krátkych intervaloch alebo pri zmene stavu.

Kedy zvoliť mesh a kedy nie

Mesh je výborný, keď potrebujeme premosťovať prekážky a nechceme, aby jeden bod vypadol. Nehodí sa však tam, kde je kritická veľmi nízka spotreba alebo kde musíme komunikáciu udržať maximálne jednoduchú; v takom prípade je lepšia hviezda s kvalitnou bránou.

4.4 Komunikačné protokoly

Výber protokolu závisí od pasívnych a aktívnych požiadaviek:

- **Wi-Fi**: vysoká priepustnosť, jednoduché pripojenie do IP sietí, vyššia spotreba — vhodné pre napájané alebo periodicky aktívne zariadenia.
- **Bluetooth Low Energy (BLE)**: nízka spotreba, krátky dosah, vhodné pre senzory a osobné IoT.
- **LoRaWAN**: veľmi dlhý dosah pri nízkej priepustnosti, vhodné pre senzory s nízkou frekvenciou odosielania v rozľahlých oblastiach.
- **Zigbee / Thread / 6LoWPAN**: mesh-priateľské protokoly pre domácu a priemyselnú automatizáciu.
- **MQTT / MQTT-SN**: lightweight publish/subscribe protokol široko používaný v IoT; umožňuje jednoduché odosielanie telemetrie a príkazov.
- **CoAP**: REST-like protokol pre constrained zariadenia, používa UDP.

4.5 Edge vs. cloud processing

Presun časti logiky na edge (blízko zariadenia) môže znížiť latenciu a prenosovú záťaž. Pre kritické rozhodovanie alebo pri potrebe súkromnosti je vhodné vykonávať spracovanie lokálne a len agregované výsledky odosielať do cloudu.

4.6 Brána (gateway) — funkcie

Brána môže plniť viacero úloh:

- protokolová konverzia (napr. LoRaWAN → MQTT),
- zabezpečenie a autentifikácia zariadení,
- lokálne procesovanie a cacheovanie dát,
- správa aktualizácií (OTA) a centralizovaná správa zariadení.

4.7 Škálovanie a správa

Pri rozsiahlom nasadení je dôležité navrhnuť adresovanie, manažment zariadení, ukladanie telemetrie a spôsob správy konfigurácií. Manažment zahŕňa monitorovanie zdravia zariadení, alarmy a plánovanie údržby.

4.8 Bezpečnostné požiadavky

Kľúčové aspekty: zabezpečenie komunikácie (TLS/DTLS), autentifikácia zariadení, bezpečné ukladanie poverení, izolácia kritických funkcií a plán aktualizácií. Pri návrhu treba myslieť na potenciálne vektory útoku a minimalizovať rozhrania vystavené priamo internetu.

4.9 Príklady nasadení

- Smart home — Wi-Fi/BLE senzory s lokálnou bránou a cloudovým dashboardom,
- Priemyselné monitorovanie — Ethernet/CAN senzory s lokálnym edge spracovaním a redundantnými bránami,
- Dálkový monitoring — LoRaWAN senzory s nízkou frekvenciou odosielania a centralizovaným backendom.

4.10 Zhrnutie

Topológia a architektúra IoT systému musí odzrkadľovať požiadavky na dosah, spoľahlivosť, bezpečnosť a spotrebu. Výber správnych nástrojov a vrstiev výrazne uľahčí škálovanie a údržbu systémov v reálnom nasadení.

5 Komunikačné protokoly

V tejto kapitole sú podrobne rozpracované protokoly a komunikačné mechanizmy, ktoré sú bežne použité v IoT systémoch. Výber konkrétneho protokolu závisí od požiadaviek aplikácie (energetická náročnosť, latencia, spoľahlivosť, šírka pásma, bezpečnosť a škálovateľnosť). Nižšie sú popísané vlastnosti, prevádzkové režimy a návrhové kompromisy pre vybrané protokoly.

5.1 MQTT

MQTT (Message Queuing Telemetry Transport) je ľahký publish/subscribe protokol postavený nad TCP/IP, optimalizovaný pre nízku šírku pásma a obmedzené zariadenia. Architektúra zahŕňa vydavateľov (publishers), odberateľov (subscribers) a sprostredkovateľa (broker).

Kľúčové vlastnosti:

- QoS (Quality of Service) úrovne: 0 (najneskôr raz), 1 (aspoň raz), 2 (presne raz) — každá úroveň prináša rozdielnú réžiu a zložitosť implementácie.
- Retained správy: broker môže uchovávať poslednú hodnotu pre dané téma a odoslať ju novým odberateľom.
- Last Will and Testament: umožňuje signalizovať náhly odchod klienta.
- Prenos nad TCP s voliteľným TLS pre zabezpečenie šifrovania a overenia identity.

Návrhové úvahy: MQTT je vhodný pre telemetriu a scenáre so sporadickými aktualizáciami; nedostatky zahŕňajú závislosť na spojenom TCP kanáli a možnú potrebu správy brokerov pre škálovanie.

5.2 CoAP

CoAP (Constrained Application Protocol) je protokol navrhnutý pre obmedzené zariadenia a siete, pracujúci nad UDP a sledujúci štýl REST (representational state transfer). Jeho základné mechanizmy zahŕňajú správy typu Confirmable (potvrdzované) a Non-confirmable, a podporu pozorovania (Observe) proaktívne zasielajúce aktualizácie.

Kľúčové vlastnosti:

- Nízka režijná réžia vďaka UDP a kompaktnému formátu hlavičky.
- Podpora metód podobných HTTP: GET, POST, PUT, DELETE.
- Možnosť zabezpečenia pomocou DTLS (Datagram TLS) alebo OSCORE (Object Security for Constrained RESTful Environments).

Návrhové úvahy: CoAP je vhodný tam, kde je potrebná REST-ová interakcia pri nízkej režii, napríklad pri komunikácii medzi senzorom a lokálnym bránovým zariadením. Vzhľadom na použitie UDP je potrebné riešiť spoľahlivosť a poradie správ aplikačne tam, kde to záleží.

5.3 HTTP/REST a WebSocket

HTTP/REST je štandardný protokol pre webové služby a často sa používa v IoT pri integrácii so servermi a cloudovými rozhraniami. Nevýhodou je vyššia režijná záťaž (TCP, hlavičky) v porovnaní s MQTT a CoAP.

WebSocket poskytuje perzistentné obojsmerné spojenie nad TCP, vhodné pre real-time dashboards a ovládanie zariadení s nízkou latenciou. Pre nízkoenergetické zariadenia sú tieto protokoly menej vhodné bez proxy alebo brány, ktorá agreguje komunikáciu.

5.4 Narrowband a LPWAN technológie: LoRaWAN, NB-IoT, Sigfox

LoRaWAN je sieťová vrstva postavená nad moduláciou LoRa, vhodná pre dlhý dosah a nízku spotrebu energie. Podporuje tri triedy zariadení (A,B,C) s rôznymi režimami prijímania, adaptívne riadenie výkonu (ADR) a režimy prispôsobenia prenosovej rýchlosti.

NB-IoT (Narrowband IoT) a Sigfox sú sieťové technológie poskytované mobilnými/operátorskými infraštruktúrami s dôrazom na nízku spotrebu, dlhú životnosť batérií a veľké pokrytie. NB-IoT poskytuje lepšiu priepustnosť a integráciu s mobilnými sieťami než Sigfox.

Návrhové úvahy: LPWAN technológie sú vhodné pre scenáre s nízkou frekvenciou zasielania údajov a veľkým dosahom, nie sú vhodné pre aplikácie s vysokými nárokmi na latenciu alebo vysokú priepustnosť.

5.5 Bluetooth Low Energy (BLE) a Zigbee

BLE poskytuje nízku spotrebu pri krátkom dosahu a model GATT (Generic Attribute Profile) na organizáciu služieb a charakteristík. BLE je vhodné pre nositeľné zariadenia a lokálne senzory, pričom nové verzie zlepšujú prenosovú rýchlosť a energetickú efektívnosť.

Zigbee je mesh-orientovaná sieťová technológia bežne používaná v domáciach automatizáciách; podporuje samoliečenie siete a distribuované smerovanie. Oba štandardy majú vlastné bezpečnostné mechanizmy (pairing, link keys), ktoré je potrebné správne implementovať.

5.6 6LoWPAN, Thread a IPv6

6LoWPAN umožňuje prenos IPv6 paketov cez nízkopásmové rádiové siete (IEEE 802.15.4), čím poskytuje možnosť priamej adresovateľnosti zariadení a používaniu štandardných protokolov IPv6. Thread stavia na 6LoWPAN a poskytuje mesh-funkcionalitu s bezpečnosťou na sieti.

5.7 Bezpečnostné aspekty protokolov

Pri navrhovaní IoT riešení je rozhodujúce zabezpečiť ochranu dát a integritu komunikácie. Základné odporúčania:

- Používať šifrovanie vrstvy transportu (TLS/DTLS) alebo aplikačné zabezpečenie (OS-CORE).
- Overovať a autorizovať zariadenia pomocou silných kľúčov alebo certifikátov; vyhýbať sa statickým, nezabezpečeným kľúčom v zariadení.
- Implementovať aktualizácie firmvéru zabezpečeným spôsobom (signed firmware, over-the-air aktualizácie s overením integrity).

5.8 Kritériá výberu protokolu

Pri rozhodovaní o voľbe protokolu je vhodné zvážiť nasledujúce kritériá: energetická náročnosť, spoľahlivosť (QoS), latencia, dostupná šírka pásma, režijné náklady, požadovaná úroveň zabezpečenia, topológia siete a požiadavky na škálovanie.

V nasledujúcich kapitolách sú tieto protokoly demonštrované na konkrétnych príkladoch a implementáciách na platforme ESP32 a prostredí MicroPython.

6 ESP32 – prehľad platformy

ESP32 predstavuje platformu obsahujúcu rodinu systémov na čipe (SoC) vyvinutých spoločnosťou Espressif Systems a široko využívaných v IoT a vstavaných aplikáciách. Charakterizuje ju bohatá sada periférií, rôzne výkonnostné a energetické profily rôznych variantov a rozvinutý ekosystém vrátane oficiálneho SDK (ESP-IDF), Arduino core a podporných implementácií ako MicroPython Systems 2026.

Klasifikácia rodín a architektúr

Platforma zahŕňa niekoľko rodín optimalizovaných pre odlišné účely: všeobecné Wi-Fi/BLE moduly (ESP32, ESP32-S3), nízko-energetické riešenia a RISC-V variácie (ESP32-C3, ESP32-C6), špecializované čipy pre IEEE 802.15.4 (ESP32-H2) a výkonné SoC pre HMI aplikácie (ESP32-P4). Architektonicky sú rozdiely dôležité pri výbere toolchainu (Xtensa vs. RISC-V), podpore periférií a pri plánovaní výkonu.

Kľúčové vlastnosti a návrhové dôsledky

Z pohľadu návrhu sú pre ESP32 relevantné nasledujúce aspekty:

- integrované rádiové subsystémy (Wi-Fi, BLE) s dosahom a spotrebou charakteristickými pre cieľovú aplikáciu;
- viacero periférií s hardvérovou podporou DMA, časovačmi a RMT pre presné časovanie;
- podpora hlbokých spánkových režimov a fine-grained power domains, čo umožňuje optimalizáciu spotreby;
- bezpečnostné mechanizmy ako secure boot, flash encryption a podpora pre OTA aktualizácie.

Návrh systému na ESP32 by mal zohľadniť približnú energetickú bilanciu (profilovanie odberu pri vysielaní Wi-Fi, pri aktívnom MCU a v spánku) a plánovať agregáciu komunikácie, aby sa minimalizovala frekvencia energeticky náročných operácií.

Vývojové nástroje a pracovná metodika

ESP-IDF poskytuje nízkoúrovňový prístup k perifériám, konfiguračný systém (menuconfig), nástroje pre profilovanie a diagnostiku. Arduino core je výhodný pre rýchle prototypovanie, PlatformIO pre integráciu do CI a MicroPython pre interaktívne experimentovanie. Pre produkčné projekty sa odporúča používať ESP-IDF spolu s robustným build systémom a testovacou infraštruktúrou.

Pokročilé techniky a best practices

Pri využití periférií je často žiaduce použiť DMA na minimalizáciu zaťaženia CPU pri prenosoch SPI/I2S alebo pri zázname ADC. Pre presné signály (WS2812, infračervené protokoly) je užitočný RMT modul namiesto softvérového bit-bangu.

Pri viacjadrových SoC (dvojjadrové varianty) je nutné riešiť synchronizáciu medzi jadrami, rozdelenie práce (napr. jadro A pre sieť, jadro B pre spracovanie), a vyhnúť sa závorom o periférie bez vhodnej synchronizácie.

Bezpečnosť a nasadenie

ESP32 podporuje zabezpečené zavádzanie (secure boot) a šifrovanie obsahu flash pamäte, čo je dôležité pre ochranu duševného vlastníctva a integritu firmvéru. Pre bezpečné nasadenie sa odporúča:

- aktivovať secure boot a flash encryption,
- používať overené metódy OTA s podpismi firmware,
- správne spravovať certifikáty a kľúče (dedikovaný secure element ak je požadované),
- minimalizovať otvorené služby a expozíciu rozhraní.

Měření výkonu a profilovanie

Pri optimalizácii systémov je vhodné merať energetickú spotrebu (napr. pomocou prúdových sond a osciloskopu) a latenciu kritických operácií. ESP-IDF obsahuje nástroje na trace a profiling, ktoré pomáhajú identifikovať úzke miesta.

Záver

ESP32 poskytuje flexibilnú platformu pre široké spektrum IoT aplikácií, avšak efektívne využitie vyžaduje pochopenie jeho interných mechanizmov — správy napájania, DMA, bezpečnostných prvkov a správnej integrácie rádiovkej vrstvy. V nasledujúcich kapitolách ukážeme tieto princípy na konkrétnych príkladoch.

7 MicroPython pre vstavané systémy

MicroPython je minimalistická implementácia jazyka Python určená pre obmedzené hardvérové platformy. Je navrhnutý tak, aby poskytoval interaktívne prostredie (REPL), priamy prístup k perifériám a rýchle prototypovanie pri zachovaní čitateľnej a expresívnej syntaxe.

Architektúra run-time a pamäťový model

MicroPython implementuje heap-based správu pamäte s garbage collection (GC). Pre návrh stabilných aplikácií je potrebné pochopiť obmedzenia dynamickej alokácie a strategicky minimalizovať vytváranie objektov v kritických slučkách. Kľúčové sú tiež mechanizmy pre "frozen" moduly (vkladanie Python kódu do firmvéru) a native moduly pre urýchlenie kritických sekcií.

Výkonnostné optimalizácie

Pre zvýšenie výkonu a deterministiky v MicroPythone sú k dispozícii viaceré techniky:

- Použitie dekorátora `micropython.native` alebo `micropython.viper` pre kompiláciu kritických funkcií do natívneho kódu.
- Vkladanie natívnych modulov v C pre časovo náročné operácie alebo pri práci s veľkými dátovými štruktúrami.
- Použitie DMA, hardvérových periférií a RMT namiesto softvérového bit-bangu.

Konkurentné modely a I/O

MicroPython podporuje viaceré modely spracovania súbežných udalostí: prerušeniami riadené handlers (ISR), asynchrónne programovanie cez `uasyncio` a jednoduché preemptívne prístupy kombinované s frontami. ISR musia byť krátke a mali by len nastavovať príznaky alebo zapisovať do fronty, kde hlavný cyklus spracuje zložitejšie akcie.

Vývoj, nasadenie a nástroje

Pre rýchle nahrávanie a ladenie sa často používa Thonny; pre produkčné buildy a integráciu sa odporúča automatizovať vytváranie firmware so zamrazenými modulmi a testovacím balíčkom. Dôležitý je aj monitoring pamäti počas vývoja a profilovanie časových kritických úsekov.

Silné a slabé stránky

MicroPython ponúka výrazné zjednodušenie vývoja, interaktívne ladenie a rýchle nasadenie. Na druhej strane, oproti natívnemu C/C++ riešeniu sú obmedzenia v rýchlosti, správe pamäte a dostupnosti knižníc. Preto pre produkčné systémy často kombinujeme MicroPython pre riadenie vyššej úrovne a natívne moduly pre výpočtovo náročné časti.

Praktické odporúčenia

- Minimalizovať dynamickú alokáciu v hlavných slučkách a v kritických handleroch.
- Používať "frozen modules" pre stabilitu a zníženie nárokov na RAM.
- Využiť `uasyncio` pre čitateľné asynchrónne I/O a riadenie viacerých úloh.
- Keď je potrebná vysoká výkonnosť, extrahovať kritické algoritmy do C rozšírení.

Nasadenie

MicroPython je vhodný pre prototypovanie, výučbu a menšie produkčné nasadenia, kde rýchlosť vývoja a flexibilita prevažujú nad maximálnou výkonnosťou. Pri väčších projektoch sa odporúča hybridný prístup: MicroPython pre orchestráciu a natívne moduly pre časovo citlivé časti.

Pre ďalšie informácie a referencie pozri MicroPython dokumentáciu a príručky k platforme ESP32 contributors 2024.

8 Príklady a experimentálne cvičenia

Táto kapitola obsahuje súbor systematicky usporiadaných príkladov a experimentálnych cvičení, ktorých cieľom je ilustrovať navrhované princípy a overiť ich v reálnom prostredí. Ku každému príkladu sú pripojené: (1) jasne definovaný cieľ, (2) krok-po-kroku implementačné pokyny, (3) skrátené úryvky kódu a (4) metodika testovania a merania výsledkov. Poznámky k implementácii zahŕňajú potenciálne chyby, predpoklady o zapojení a odporúčania pre rozšírenie alebo zmeny konfigurácie.

9 Servisný test dosky

Tento príklad slúži ako rýchly self-test po flashnutí zariadenia. Cieľom je overiť základné periférie (displej, LED, senzory, tlačidlá, bzučiak) a získať rýchlu spätnú väzbu o tom, či sú piny a moduly správne zapojené. V texte uvádzam, ktoré merania sú deterministické (napr. ADC čítanie) a ktoré závisia na externých podmienkach (napr. svetelnosť), ako aj tipy na ladenie (kontrola napájania, sériový výstup, jednoduché multimeter kontroly).

Mapovanie pinov a zber výsledkov

Najprv si nadefinujem piny, ktoré program používa, aby bolo jednoduché upraviť zapojenie podľa konkrétnej dosky. Zoznam `results` slúži ako jednotný formát pre neskoršie zobrazenie výsledkov a uľahčuje rozšírenie testov alebo odoslanie reportu cez sériový port.

```
1 from machine import Pin, PWM, I2C, ADC
2 import time
3
4 PIN_I2C_SDA = 1
5 PIN_I2C_SCL = 2
6 PIN_RGB_R = 7
7 PIN_RGB_G = 5
8 PIN_RGB_B = 3
9 results = []
```

Pomocné funkcie pre výsledky

Pomocné funkcie izolujú opakujúcu sa logiku: `add_result()` formátuje a ukladá výsledok testu (názov, stav, poznámka), čo zjednoduší tlač správy alebo export. `ask_yes_no()` sa používa na manuálne overenie vizuálnych komponentov (napr. či OLED skutočne zobrazuje text) a ukážem, prečo je vhodné mať takýto blok oddelene pre jednoduché testovanie.

```
1 def add_result(name, ok, note=""):
2     state = "PASS" if ok else "FAIL"
3     results.append((name, state, note))
4
5 def ask_yes_no(question):
6     while True:
7         ans = input(question + " [y/n]: ").strip().lower()
8         if ans in ("y", "yes", "a", "ano"):
9             return True
10        if ans in ("n", "no", "nie"):
11            return False
```

Test OLED, RGB LED a WS2812

V tejto časti sa zameriame na vizuálnu spätnú väzbu: krátky text na OLED potvrdí komunikáciu po I2C, diskretná RGB LED overí základnú logiku výstupov a WS2812 reťazec demonštruje adresovateľné LED a správne časovanie dátového signálu. Poznámka: pri WS2812 je kritické správne napájanie a dostatočné časovanie; v texte spomínam ako testovať po jednom pixeli a na čo si dávať pozor pri viacnásobnom píne.

```
1 def test_oled():
2     i2c = I2C(0, sda=Pin(PIN_I2C_SDA), scl=Pin(PIN_I2C_SCL), freq
3         =400000)
4     oled = ssd1306.SSD1306_I2C(128, 64, i2c)
5     oled.text("SELF TEST", 0, 12)
6     oled.show()
7
8 def test_rgb_discrete():
9     r = Pin(PIN_RGB_R, Pin.OUT)
10    g = Pin(PIN_RGB_G, Pin.OUT)
11    b = Pin(PIN_RGB_B, Pin.OUT)
12
13 def test_ws2812():
14     np = neopixel.NeoPixel(Pin(PIN_LED_DATA), WS2812_COUNT)
15     np[0] = (255, 0, 0)
16     np.write()
```

Test bzučiaka, tlačidiel, fotorezistora a DHT

Tu prichádzajú testy, ktoré zahŕňajú časovanie (bzučiak), čítanie digitálnych vstupov (tlačidlá), analógové meranie (fotorezistor cez ADC) a sekvenčné meranie (DHT senzor). V texte popisujem očakávané rozsahy hodnôt, ako si nastaviť debounce pre tlačidlá a prečo pri DHT senzore potrebujeme čakať medzi meraniami.

```
1 def test_buzzer():
2     bz = PWM(Pin(PIN_BUZZER), freq=1000, duty=0)
3     for f in (523, 659, 784):
4         bz.freq(f)
5         bz.duty(512)
6
7 def test_buttons():
8     buttons = [("USR1", Pin(PIN_BTN_1, Pin.IN)), ...]
9
10 def test_photo():
11     adc = ADC(Pin(PIN_PHOTO))
12     v = adc.read()
13
14 def test_dht():
15     sensor = dht.DHT22(Pin(PIN_DHT))
16     sensor.measure()
```

Spustenie testov a súhrn

V hlavnej časti je cyklus, ktorý postupne vykoná jednotlivé testy a zapíše výsledky do 'results'. Toto miesto je ideálne na doplnenie logovania do súboru alebo na odosielanie výsledkov cez sériový port/HTTP. V texte tiež navrhujem, ako pridať timeouty a opätovné pokusy pre nestabilné periférie.

```
1 for fn in (test_oled, test_rgb_discrete, test_ws2812,  
2           test_buzzer, test_buttons, test_photo, test_dht):  
3     fn()  
4  
5 print("=== SUMAR ===")  
6 for name, state, note in results:  
7     print(name, state, note)
```

10 Metodika experimentov a hodnotenie

Pre vykonávanie reprodukovateľných experimentov a porovnateľné hodnotenie výsledkov odporúčame nasledovné postupy:

- Definovať jasné metriky (spotreba energie, latencia odozvy, chybovosť, úspešnosť prenosu) a meracie podmienky (teplota, napájacie napätie).
- Použiť opakované merania a štatistické spracovanie (priemer, štandardná odchýlka), aby sa rozlíšili náhodné od systematických chýb.
- Pri meraní spotreby energie zaznamenať prúdové profily počas celého cyklu, najmä pri spúšťaní rádiových modulov.
- Pri sieťových testoch vykonávať testy v rôznych podmienkach (rôzne kanály, vzdialenosti, rušenie) a zaznamenať metriky úspešnosti prenosu a latencie.
- Dokumentovať verzie firmvéru, konfiguráciu periférií a použité knižnice, aby bolo možné experimenty reprodukovateľ.

Tieto zásady zlepšujú kvalitu meraní a uľahčujú následnú analýzu pri porovnávaní návrhov alebo pri optimalizácii parametrov systému.

11 Interakcia s tlačidlami: hra hľadania

Tento príklad demonštruje praktické použitie tlačidiel ako vstupu pre jednoduchú hru, pričom sa rieši detekcia stlačení, tvorba pseudo-náhody z kombinácie stlačení a spätná väzba užívateľovi. V texte uvádzam, ako spracovať bouncing, prečo nie je vhodné použiť ťažký výpočet v IRQ a ako testovať, či sú piny správne priradené.

Tlačidlá a základné konštanty

V tejto fáze priradíme fyzické piny k logickým menám, čo zvyšuje čitateľnosť kódu a uľahčuje zmenu zapojenia. Tiež vysvetľujem, kedy treba aktivovať vnútorné pull-up alebo pull-down rezistory a čo očakávať, keď je logická úroveň invertovaná (závisí to od zapojenia tlačidla).

```
1 from machine import Pin
2 import time
3
4 BTN1 = Pin(39, Pin.IN)
5 BTN2 = Pin(37, Pin.IN)
6 BTN3 = Pin(35, Pin.IN)
7 BTN4 = Pin(33, Pin.IN)
```

Generovanie náhodného čísla z tlačidiel

Tento jednoduchý spôsob generovania náhody čerpá entropiu zo stavu tlačidiel pri štarte (alebo z počtu stlačení). Vysvetlím, prečo to postačuje pre jednoduchú hru, ale nie je vhodné pre krypto-grafické účely, a ako by sa dalo rozšíriť o ďalšie zdroje náhody (ADC šum, čas medzi stlačeniami).

```
1 def get_seed():
2     seed = 0
3     if BTN1.value() == 0:
4         seed += 1
5     if BTN2.value() == 0:
6         seed += 2
7     return seed
```

Herný cyklus

V hlavnej slučke program číta vstup od používateľa, porovnáva ho s tajným číslom a dáva priebežnú spätnú väzbu. Popisujem možnosti, ako pridať časový limit, skórovací systém alebo vizuálnu odozvu cez LED, a tiež navrhujem, ako spraviť vstup robustnejším (validácia, ošetrenie chýbného vstupu).

```
1 while True:
2     guess = int(input("Tipni cislo 1-4: "))
3     if guess == secret:
4         print("Spravne")
5         break
6     print("Skus znova")
```

12 Digitálne riadenie RGB LED

Ukážka ukazuje najzákladnejší prístup k ovládaniu farieb pomocou samostatných digitálnych výstupov. Vysvetľujem, kedy je tento režim dostačujúci (indikácia stavov) a kedy treba použiť PWM pre plynulé ladenie jasu.

Digitálne výstupy RGB LED

```
1 from machine import Pin
2 import time
3
4 red = Pin(7, Pin.OUT)
5 green = Pin(5, Pin.OUT)
6 blue = Pin(3, Pin.OUT)
```

Postupné prepínanie farieb

Tento jednoduchý cyklus demonštruje, ako kombinovať kanály pre základné farby. Upozorňujem na potrebu dobrého riadenia prúdového limitu a na to, že pri viacerých kanáloch sú súčty prúdov dôležité pre napájanie dosky.

```
1 while True:
2     red.value(1)
3     green.value(0)
4     blue.value(0)
5     time.sleep(0.3)
```

13 PWM riadenie jasú RGB LED

V tejto kapitole prechádzame k jemnejšiemu ovládaniu pomocou PWM. Vysvetlím vzťah medzi frekvenciou PWM a flicker-om, typické rozsahy pre ESP32 a dôvody, prečo mať rovnomerné rozdelenie časových kvant pre jednotlivé kanály.

PWM kanály pre RGB LED

Každá farba má vlastný PWM kanál.

```
1 from machine import Pin, PWM
2
3 r = PWM(Pin(7), freq=1000)
4 g = PWM(Pin(5), freq=1000)
5 b = PWM(Pin(3), freq=1000)
```

Nastavenie hodnôt jasú

Funkcia prijíma hodnoty jasú v očakávanom intervale a mapuje ich na duty cycle PWM. Popíšem konverziu rozsahov (napr. 0–255 na 0–1023) a tipy, ako zabrániť preťaženiu GPIO pri náhlom zvýšení jasú.

```
1 def set_rgb(r_val, g_val, b_val):
2     r.duty(r_val)
3     g.duty(g_val)
4     b.duty(b_val)
```


14 Prerušená a meranie dĺžky stlačenia

Ukážka ilustruje použitie prerušenia (IRQ) na efektívnu detekciu stlačenia bez náročného polling-u v hlavnej slučke. V texte rozoberiem debounce stratégie, bezpečnosť pri práci s globálnymi premennými v IRQ kontexte a prečo by sa v obsluhu prerušenia nemali volať pomalé funkcie.

Stav a časovanie pre IRQ

```
1 from machine import Pin, I2C
2 import time
3
4 press_started = None
5 last_duration = 0
```

IRQ obsluha s debounce

IRQ handler by mal byť krátky: iba zaznamenať čas začiatku alebo konca stlačenia a nastaviť príznak. Vysvetlím, ako merať čas pomocou `time.ticks_ms()` a prečo je bezpečnejšie spracovať dĺžku tlačenia v hlavnej slučke, nie priamo v handleri.

```
1 def make_irq_handler(name):
2     def handler(pin):
3         global press_started, last_duration
4         if pin.value() == 0:
5             press_started = time.ticks_ms()
6         else:
7             last_duration = time.ticks_diff(time.ticks_ms(),
8                                             press_started)
9     return handler
```

Pripojenie handlerov na tlačidlá

```
1 btn1 = Pin(39, Pin.IN)
2 btn1.irq(trigger=Pin.IRQ_FALLING | Pin.IRQ_RISING,
3          handler=make_irq_handler("USR1"))
```

Zobrazenie posledného stlačenia

Program priebežne ukazuje poslednú nameranú dĺžku stlačenia.

```
1 while True:
2     print("Posledne stlacenie:", last_duration, "ms")
3     time.sleep(0.2)
```

15 Sériové LED WS2812B

Adresovateľný LED pásik (WS2812B) ilustruje protokol, kde sú dáta odosielané v presných časových oknách. Popíšem obmedzenia prenosu, potrebu presného nastavenia času a rady na napájanie viacerých pixelov (spoločná zem, kapacitné vyhladenie napätia).

NeoPixel objekt pre WS2812B

```
1 from machine import Pin
2 import neopixel
3
4 np = neopixel.NeoPixel(Pin(4), 12)
```

Nastavenie farieb

Vysvetlím, ako funguje buffer pixelov a prečo je dôležité volať `np.write()` až po nastavení všetkých hodnôt. Spomeniem tiež, že niektoré implementácie vyžadujú kratšie oneskorenia pred a po zápise dát.

```
1 def set_all(color):
2     for i in range(12):
3         np[i] = color
4     np.write()
```

16 PWM bzučiak a krátka melódia

Ukážka demonštruje generovanie tónov pomocou PWM, kde sa mení frekvencia namiesto duty-cyklu. Popisujem, ako nastaviť vhodnú duty pre čistý tón, prečo by sa nemal používať maximálny výkon nepretržite a ako zložiť jednoduchú melódiu bez externých knižníc.

PWM výstup pre bzučiak

```
1 from machine import Pin, PWM
2 import time
3
4 buzzer = PWM(Pin(11), freq=1000, duty=0)
```

Funkcia na prehratie tónu

Funkcia nastaví frekvenciu a krátko aktivuje výstup. V poznámke uvádzam, že pri dlhých tónoch je vhodné znižovať duty alebo vložiť malé prestávky, aby sa zabránilo prehrievaniu aktívneho bzučiaka.

Krátka melódia

```
1 for note in (523, 659, 784):
2     beep(note, 180)
3     time.sleep_ms(60)
```

17 Analógové meranie svetla

V tejto kapitole popisujem meranie odporu fotorezistoru cez ADC a postup kalibrácie, ktorý zabezpečí zmysluplné percentuálne hodnoty v rôznych svetelných podmienkach. Upozorním na offsety ADC, potrebu filtrovania a ako zmerať minimálne a maximálne hodnoty prakticky.

Inicializácia ADC

```
1 from machine import Pin, ADC
2
3
4 adc.atten(ADC.ATTN_11DB)
```

Kalibrácia minima a maxima

Kalibrácia spočíva v pokrývaní extrémov (tma vs. silné svetlo) a zapísaní týchto hodnôt, ktoré sa potom použijú na normalizáciu vstupu. V texte dávam tipy, ako vykonať viacnásobné merania a použiť klzavý priemer pre stabilnejší výsledok.

```
1 min_v = 65535
2 max_v = 0
3 v = adc.read()
```

Prepočet na percentá

Normalizácia spraví z ADC čísla ľahšie čitateľný výstup.

```
1 def normalize(value, min_v, max_v):
2     span = max_v - min_v
3     return int((value - min_v) * 100 / span)
```

18 Senzor DHT: teplota a vlhkosť

Krátka, ucelená ukážka načítania teploty a vlhkosti zo senzora DHT. Okrem základného princípu merania popisujem časové obmedzenia medzi meraniami a prípadné chyby pri čítaní, ktoré treba ošetriť (timeouty, chybové kódy).

```
1 from machine import Pin
2 import dht
3 import time
4
5 sensor = dht.DHT22(Pin(18))
6
7 while True:
8     sensor.measure()
9     print(sensor.temperature(), sensor.humidity())
10    time.sleep(2)
```

19 Wi-Fi prístupový bod a webové ovládanie

Tento príklad prepája lokálne periférie s jednoduchým webovým rozhraním cez vlastný access point. Popisujem architektúru: inicializácia periférií, tvorba HTML rozhrania pre ovládanie, bezpečnostné a spoľahlivé spracovanie požiadaviek, a návrhy na rozšírenie (autentifikácia, stavové stránky, AJAX pre aktualizácie bez reloadu).

1. Pripravím periférie a fallbacky

V tejto časti sa oboznámime so spôsobom, ako robustne inicializovať periférie: najprv skúsiť importovať voliteľné moduly a v prípade chýb poskytnúť fallback (napr. simulované čítania). Uvádzam príklady, čo robiť, keď senzor nie je prítomný, aby server neskončil s chybou.

```
1 import socket, time, network
2 from machine import Pin, PWM, I2C
3
4 try:
5     import dht
6 except ImportError:
7     dht = None
```

2. Vytvorím pomocné funkcie pre výstupy

Oddelené funkcie pre ovládanie výstupov zlepšujú čitateľnosť a uľahčujú viazanie rozhrania (mapovanie tlačidiel na akcie). Vysvetlím, ako mať čisté API pre zmenu stavu (napr. `set_rgb(name)`) a prečo je dobré mať centrálny stav zariadenia, ktorý HTML stránka vie prečítať.

```
1 def set_rgb(name):
2     colors = {"red": (1, 0, 0), "green": (0, 1, 0), "off": (0, 0, 0)}
3     r, g, b = colors.get(name, (0, 0, 0))
4     red.value(r)
5     green.value(g)
6     blue.value(b)
```

3. Vygenerujem HTML stránku

HTML stránka je minimálna a slúži na demonštráciu ovládania: tlačidlá odosielaajú query parametre a stránka zobrazuje aktuálne merania. Uvádzam, ako zaradiť CSRF bezpečnosť minimalizovaným spôsobom (jednorazové tokeny) a ako vylepšiť používateľské rozhranie jednoduchým CSS/JS.

```
1 def html_page(temp, hum):
2     return """<!doctype html>
3     <html>
4     <body>
5     <a href='/?rgb=red'>Red</a>
6     <a href='/?buzz=1'>Pipni</a>
7     </body></html>"""
```

4. Spustím access point

Spustenie AP modeluje režim, keď zariadenie poskytuje vlastnú sieť bez potreby existujúcej infraštruktúry. Popisujem, kedy je vhodné AP vs. STA režim, ako nastaviť jednoduché heslo a čo očakávať pri obmedzenom dosahu/kanálových konfliktoch.

```
1 def start_ap():
2     ap = network.WLAN(network.AP_IF)
3     ap.active(True)
4     ap.config(essid=AP_SSID, password=AP_PASSWORD, authmode=3)
```

5. Obslúžim HTTP požiadavky

Server prijíma jednoduché GET požiadavky a na základe parametrov vykonáva akcie (zapnúť LED, prehriať bzučiak). Upozorňujem na potrebu parsovania a validácie parametrov, správnu uzávierku soketov a na to, že server by mal byť tolerantný k neúplným požiadavkám.

```
1 while True:
2     conn, _ = s.accept()
3     req = conn.recv(1024)
4     if "rgb" in req:
5         set_rgb(req["rgb"])
6     if req.get("buzz") == "1":
7         buzz()
```

20 Odosielanie dát do ThingSpeak

Praktický príklad integrácie so vzdialenou cloudovou službou. Popisujem, ako efektívne bufferovať merania, zaistiť opakované odosielanie pri výpadkoch siete a minimalizovať spotrebu energie pri pravidelnom odosielaní dát.

1. Pripravím Wi-Fi a senzor

```
1 import network, time, urequests
2 from machine import Pin
3 import dht
4
5 sensor = dht.DHT22(Pin(18))
```

2. Pripojím sa do siete

Keď sa Wi-Fi pripojenie nepodarí, program to vie zistiť a skúsiť znova.

```
1 def wifi_connect():
2     wlan = network.WLAN(network.STA_IF)
3     wlan.active(True)
4     wlan.connect(WIFI_SSID, WIFI_PASSWORD)
```

3. Načítam hodnoty a pošlem ich

ThingSpeak očakáva jednoduchý HTTP request s parametrami kanála.

```
1 def thingspeak_update(temp, hum):
2     url = THINGSPEAK_URL + "?field1=%s&field2=%s" % (temp, hum)
3     urequests.get(url).close()
```

4. Opakujem cyklus v hlavnej slučke

```
1 while True:
2     sensor.measure()
3     thingspeak_update(sensor.temperature(), sensor.humidity())
4     time.sleep(20)
```


21 MQTT publish/subscribe komunikácia

MQTT je vhodný tam, kde zariadenie musí nielen odosielať dáta, ale aj prijímať príkazy. Tento príklad ukazuje oba smery komunikácie.

1. Pripravím Wi-Fi a MQTT konfiguráciu

```
1 import network, time
2 from umqtt.simple import MQTTClient
3
4 CLIENT_ID = b"iotstich-01"
5 MQTT_BROKER = b"test.mosquitto.org"
6 TOPIC_PUB = b"iotstich/telemetry"
7 TOPIC_SUB = b"iotstich/cmd"
```

2. Pripojím sa do Wi-Fi

```
1 def wifi_connect():
2     wlan = network.WLAN(network.STA_IF)
3     wlan.active(True)
4     wlan.connect(WIFI_SSID, WIFI_PASSWORD)
```

3. Definujem MQTT callback

Callback reaguje na prichádzajúce správy a vie podľa nich zmeniť stav dosky.

```
1 def on_message(topic, msg):
2     print(topic, msg)
3     if msg == b"ping":
4         print("pong")
```

4. Publikujem telemetriu v hlavnej slučke

```
1 client = MQTTClient(CLIENT_ID, MQTT_BROKER)
2 client.set_callback(on_message)
3 client.connect()
4 client.subscribe(TOPIC_SUB)
5
6 while True:
7     client.publish(TOPIC_PUB, b"25.4,60.1")
8     client.check_msg()
9     time.sleep(10)
```

Referencie

contributors, MicroPython (2024). *MicroPython Documentation*. <https://docs.micropython.org>.
Systems, Espressif (2026). *ESP32 SoCs*. <https://www.espressif.com/en/products/socs>.